October 13, 2026

Surrogate Models for Fast Apogee Prediction

Application to On-Board Flight Control Algorithms

Marco D'Agostino

Case study developed within Skyward Experimental Rocketry
(Politecnico di Milano)

Revision: *2*

Released with the `kriging-surrogate` repository under the MIT License.

Abstract

This report documents the development and validation of a Kriging surrogate model for real-time apogee prediction in rocket flight control. Traditional methods for apogee estimation either carry the computational cost of numerical Ordinary Differential Equation (ODE) integration, incompatible with the on-board per-cycle budget, or rely on pre-computed lookup tables, whose memory footprint grows steeply with the number of input variables. The approach followed here combines trajectory-based sampling of stochastic 6-DOF flight simulations with Gaussian Process regression: the resulting predictor maps a six-dimensional flight state to apogee altitude fast enough for the on-board control loop, and returns a variance with each prediction, which the control law can use as a confidence indicator.

Model accuracy is assessed with two complementary metrics: the test error, which measures fidelity to the underlying ODE dynamics, and the validation error against independent stochastic simulations whose trajectories include effects the model does not take as inputs — angular rates, full three-component velocity, non-zero wind, parametric aerodynamic uncertainties — and which therefore also absorbs the gap between the ballistic training target and the full six-DOF apogee. On held-out data the surrogate attains an RMSE of 0.77 m (0.39 m mean absolute error), and under stochastic validation an RMSE of 6.01 m. A prediction costs a few tens of microseconds (warmed-up median over batched queries), orders of magnitude below the order-of-a-second cost of a full ascent simulation. A design-space study shows the reference model is not the only viable size: two smaller configurations match its validation accuracy to within a few percent at a fraction of the cost, trading loop time against memory footprint on the target hardware.

The model is intended to support Skyward's engine-shutdown and air-brake control algorithms. Its pointwise variance is empirically optimistic by roughly 1.5× (Section 3.5.3), so the raw uncertainty bands are best read as relative confidence indicators, with the quantified inflation factor applied before using them as absolute bounds.

Contents

Acronyms **1**

Notation **2**

1 Problem Definition and Context **4**

1.1 Motivation and Application Context 4

1.1.1 Real-Time and Embedded System Constraints 4

1.1.2 Motivation for Data-Driven Surrogates 4

1.2 Mathematical Formulation 5

1.2.1 Flight Dynamics Model 5

1.2.2 Target Output: Apogee Estimation 6

1.2.3 Input Features and State Space 6

1.2.4 Reduced Feature Vector for Surrogate Modeling 6

1.2.5 Rationale for Feature Selection 7

2 Kriging-Based Surrogate Modeling Architecture **8**

2.1 Overview of Kriging 8

2.1.1 Gaussian Process Formulation 8

2.1.2 Kriging Predictor and Variance 9

2.2 Hyperparameter Estimation 10

2.2.1 Concentrated Log-Likelihood 10

2.2.2 Optimizer Selection 10

2.3 Computational Considerations 11

2.3.1 Training Complexity 11

2.3.2 Prediction Complexity 12

2.4 Sampling Strategy 12

2.4.1 Input Domain Definition via Stochastic Simulations 12

2.4.2 Training Sample Extraction 13

3 Results and Performance Analysis **15**

3.1 Nominal Kriging Model: Training and Hyperparameter Selection 15

3.1.1 Training Configuration 15

3.1.2 Optimized Hyperparameters 15

3.1.3 Dataset Convergence: Sample Size Selection 15

3.1.4 Model Sensitivity Study 17

3.1.5 Isolating the Two-Sample Effect from Training-Set Size 18

3.2 Accuracy Analysis 20

3.2.1 Test Set Performance Metrics . . . 20
3.2.2 Prediction Error Distribution . . . 20
3.2.3 Parity Plots . . . 20
3.2.4 Surrogate Response Maps . . . 21
3.3 Robustness Under Parametric Uncertainty . . . 21
3.3.1 Validation Against Stochastic Simulations . . . 21
3.4 Computational Performance . . . 24
3.4.1 Training Cost . . . 24
3.4.2 Inference Cost . . . 24
3.4.3 Design-Space Trade-off and Deployment Recommendation . . . 25
3.5 Sensitivity and Uncertainty Analysis . . . 26
3.5.1 Feature Importance via Gradient-Based Analysis . . . 26
3.5.2 Prediction Uncertainty Map . . . 27
3.5.3 Variance Calibration . . . 27

4 Conclusions and Future Perspectives **29**
4.1 Summary and Key Results . . . 29
4.2 Implications for the Flight Software . . . 29
4.3 Limitations and Caveats . . . 30
4.4 Future Work . . . 30

Acronyms

SDA Shutdown Algorithm
ABK Airbrakes
ODE Ordinary Differential Equation
RAM Random Access Memory
GP Gaussian Process
BLUP Best Linear Unbiased Predictor
LOOCV Leave-One-Out Cross-Validation
SQP Sequential Quadratic Programming
RMSE Root Mean Squared Error
MAE Mean Absolute Error

Notation

Frames and vectors

$\mathbf{v}_B = [u\ v\ w]^\top$	Translational velocity in body axes
$\boldsymbol{\omega}_B = [p\ q\ r]^\top$	Angular velocity in body axes
$\mathbf{F}_B$, $\mathbf{M}_B$	Total external force and aerodynamic moment about the centre of mass, body axes
$\mathbf{F}_{aero,B}$, $\mathbf{F}_{g,B}$	Aerodynamic and gravity force vectors in body axes
$\mathbf{R}_{NED\to B}$	Direction cosine matrix from inertial to body frame
$\mathbf{g}_{NED}$	Gravity vector in the inertial frame

Attitude

ϕ, θ, ψ	Roll, pitch and yaw Euler angles (3–2–1 sequence)
$\mathbf{q}$	Unit quaternion representing attitude
$\mathbf{T}_{321}(\phi, \theta)$	Kinematic matrix relating body rates (p, q, r) to Euler angle rates

Inertia and mass

m	Rocket mass (evaluated at prediction time when used as a feature)
$\mathbf{I}_B$	Inertia tensor about the centre of mass, body axes

Aerodynamics

q_∞, S, L	Dynamic pressure, reference area and reference length
C_X, C_Y, C_Z	Force coefficients in body axes
C_l, C_m, C_n	Moment coefficients about the centre of mass, body axes
α_{tot}, M, h	Total angle of attack, Mach number and altitude
δ_{abk}	Airbrake deflection

States and features

$\mathbf{s}_0$	Full state vector at prediction time
$\mathbf{x}$	Reduced feature vector used by the surrogate
z_0, $v_{z,0}$	Altitude and vertical velocity at prediction time
$v_{h,0} = \sqrt{v_{x,0}^2 + v_{y,0}^2}$	Horizontal velocity magnitude at prediction time
h_f	Apogee altitude (maximum altitude reached)
$\mathcal{F}$	High-fidelity apogee mapping, $h_f = \mathcal{F}(\mathbf{x})$

Kriging model

$\mathbf{R}$	Correlation matrix, $R_{ij} = R(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$
$\mathbf{r}(\mathbf{x}_*)$	Correlation vector between a query point and the training points
θ	Correlation parameters (inverse length-scales)
$\mu(\mathbf{x}) = \mathbf{f}(\mathbf{x})^T\boldsymbol{\beta}$	Regression trend, with polynomial basis $\mathbf{f}$ and coefficients $\hat{\boldsymbol{\beta}}$ estimated by generalized least squares
$\hat{\sigma}^2$	Estimated process variance
$\mathcal{L}$	Loss function for hyperparameter optimization

Sample counts

N_{sim}	Number of stochastic 6-DOF ascent simulations in the campaign
N_{traj}	Number of trajectories, split by trajectory into train and test (subscripts $N_{traj,train}$, $N_{traj,test}$)
N_{pts}	Post-cutoff samples drawn per trajectory
n	Kriging training set size, $n = N_{traj,train} \times N_{pts}$
n_{test}	Test set size, $n_{test} = N_{traj,test} \times N_{pts}$
n_{val}	Validation set size: one true 6-DOF apogee per held-out test trajectory, independent of N_{pts}
d	Input feature dimension
n_β	Regression trend basis width

Chapter 1

Problem Definition and Context

1.1 Motivation and Application Context

The primary objective of the flight control system of Hydra, a liquid-propellant sounding rocket developed by Skyward Experimental Rocketry, is to achieve a target apogee altitude with high precision, minimizing overshoot and undershooting which can compromise mission success and safety margins. This goal is pursued through two complementary control strategies: the Shutdown Algorithm (SDA) engine-shutdown logic and Airbrakes (ABK) deployment. Both need a fast and accurate apogee estimate at every control cycle.

1.1.1 Real-Time and Embedded System Constraints

The flight control algorithms execute on the onboard flight computer within a fixed control loop, so any apogee predictor must respect a tight per-cycle computational budget:

- **Prediction latency:** apogee estimation and command generation must complete well within a single control cycle, leaving only a small fraction of the available time for the predictor itself;
- **Memory constraints:** the flight computer has limited Random Access Memory (RAM), which precludes large lookup tables or heavy numerical solvers;
- **Deterministic execution:** code should run with a fixed memory footprint and no dynamic allocation to preserve real-time guarantees;
- **Limited compute:** the embedded processor cannot afford expensive online operations such as repeated matrix factorizations or iterative nonlinear solvers.

As a concrete reference point, one prediction costs a few tens of microseconds; the measurement protocol and the full timing breakdown are reported in Chapter 3.

1.1.2 Motivation for Data-Driven Surrogates

Given these constraints, the practical candidates are closed-form energy balances, pre-computed tables and data-driven surrogates. This work develops a Kriging-based **surrogate model**, without attempting a head-to-head comparison against the other options [1, 2]. The choice rests on a combination of properties. Evaluating a trained Gaussian Process (GP) predictor costs $\mathcal{O}(n)$ operations for the mean and $\mathcal{O}(n^2)$ for the variance, with n the number of training samples, and that variance comes with every prediction, enabling probabilistic control strategies that account for model fidelity. By fitting directly to high-fidelity ODE simulations, the surrogate tracks the underlying physics with an approximation error quantified on held-out data (Chapter 3) while avoiding the cost of re-integration. Finally, memory grows with the number of training samples rather than exponentially with the input dimension, as a dense table's does, so the approach extends to higher-dimensional inputs (wind, attitude uncertainties) if needed. The Kriging implementation used throughout this report is released as the open-source MATLAB library `kriging-surrogate`.

1.2 Mathematical Formulation

1.2.1 Flight Dynamics Model

During the coasting phase (after main engine cutoff), the rocket is modeled as a rigid body with constant inertia and no thrust. The equations of motion are written in body axes.

The translational dynamics in body axes are

$$m\dot{\mathbf{v}}_B + \boldsymbol{\omega}_B \times (m\mathbf{v}_B) = \mathbf{F}_{aero,B} + \mathbf{F}_{g,B}, \tag{1.1}$$

where $\mathbf{v}_B = [u\ v\ w]^T$ is the translational velocity vector in body axes, $\boldsymbol{\omega}_B = [p\ q\ r]^T$ is the angular velocity vector in body axes, $\mathbf{F}_{aero,B}$ is the aerodynamic force vector in body axes and $\mathbf{F}_{g,B}$ is the gravity force expressed in body axes.

The gravity force in body axes is obtained by rotating the inertial gravity vector:

$$\mathbf{F}_{g,B} = \mathbf{R}_{NED\to B}\, m\mathbf{g}_{NED}, \tag{1.2}$$

where $\mathbf{g}_{NED} = [0\ 0\ g]^T$ is the gravity vector in the inertial frame and $\mathbf{R}_{NED\to B}$ is the direction cosine matrix from inertial to body frame.

The rotational dynamics about the center of mass are

$$\mathbf{I}_B\dot{\boldsymbol{\omega}}_B + \boldsymbol{\omega}_B \times (\mathbf{I}_B\boldsymbol{\omega}_B) = \mathbf{M}_B, \tag{1.3}$$

where $\mathbf{I}_B$ is the constant inertia tensor in body axes and $\mathbf{M}_B$ is the aerodynamic moment vector about the center of mass, also expressed in body axes.

Attitude is represented internally using unit quaternions $\mathbf{q}$ (not to be confused with the pitch rate q), while Euler angles (ϕ, θ, ψ) are used only for feature definition. Quaternion kinematics are

$$\dot{\mathbf{q}} = \tfrac{1}{2}\,\Omega(\boldsymbol{\omega}_B)\,\mathbf{q}, \tag{1.4}$$

where $\Omega(\boldsymbol{\omega}_B)$ is the standard kinematic matrix built from (p, q, r).

The aerodynamic forces and moments about the centre of mass, in body axes, are

$$\mathbf{F}_{aero,B} = q_\infty S \begin{bmatrix} -C_X(\alpha_{tot}, \phi, M, h, \delta_{abk}) \\ C_Y(\alpha_{tot}, \phi, M, h, \delta_{abk}) \\ -C_Z(\alpha_{tot}, \phi, M, h, \delta_{abk}) \end{bmatrix}, \tag{1.5}$$

$$\mathbf{M}_B = q_\infty S L \begin{bmatrix} C_l(\alpha_{tot}, \phi, M, h, \delta_{abk}) \\ C_m(\alpha_{tot}, \phi, M, h, \delta_{abk}) \\ C_n(\alpha_{tot}, \phi, M, h, \delta_{abk}) \end{bmatrix}. \tag{1.6}$$

where q_∞ is the dynamic pressure, S the reference area, L a reference length, α_{tot} the total angle of attack, M the Mach number, h the altitude and ϕ the roll angle. Both force and moment coefficients are defined with respect to the rocket center of mass.

For later use in feature definition, the Euler angle rates are related to body rates through the 3–2–1

kinematic matrix:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \mathbf{T}_{321}(\phi, \theta) \begin{bmatrix} p \\ q \\ r \end{bmatrix}, \tag{1.7}$$

where $\mathbf{T}_{321}$ is the standard 3–2–1 (yaw–pitch–roll) Euler rate matrix.

1.2.2 Target Output: Apogee Estimation

The high-fidelity apogee altitude is defined as the maximum altitude reached by the rocket when integrating the full six-degree-of-freedom dynamics from a given initial state until vertical velocity becomes zero:

$$h_f = \max_{t \in [t_0, t_{end}]} h(t), \tag{1.8}$$

where $h(t)$ is the altitude obtained by numerically integrating Equations (1.1),(1.3) and (1.4).

For a fixed vehicle geometry and nominal atmospheric model, the apogee can be viewed as a deterministic function of the initial state and control inputs:

$$h_f = \mathcal{F}(\mathbf{x}), \tag{1.9}$$

where $\mathbf{x} \in \mathbb{R}^d$ collects the initial conditions at the time of prediction (e.g., at engine cutoff). In practice, the function $\mathcal{F}$ also depends on geometric and atmospheric parameters through the aerodynamic coefficients and wind model. These dependencies are kept fixed in the nominal dataset used to train the surrogate and are later relaxed in the validation analysis with stochastic simulations.

1.2.3 Input Features and State Space

The full state vector at the prediction time can be written as

$$\mathbf{s}_0 = \begin{bmatrix} \mathbf{r}_0^\top & \mathbf{v}_0^\top & \phi_0 & \theta_0 & \psi_0 & p_0 & q_0 & r_0 & m & \delta_{abk} \end{bmatrix}^\top, \tag{1.10}$$

where $\mathbf{r}_0$ is the position (e.g., in NED coordinates), $\mathbf{v}_0$ the velocity vector in body coordinates, $(\phi_0, \theta_0, \psi_0)$ the Euler angles, (p_0, q_0, r_0) the body angular rates, m the mass, and δ_{abk} the airbrake deflection at the prediction time.

In principle, the apogee function $\mathcal{F}$ depends on all components of $\mathbf{s}_0$ as well as on atmospheric wind and geometric parameters. However, for onboard implementation the input dimension must be reduced to variables that are both influential and reliably measurable in flight.

1.2.4 Reduced Feature Vector for Surrogate Modeling

The surrogate model does not operate on the full state $\mathbf{s}_0$ but on a reduced feature vector

$$\mathbf{x} = \begin{bmatrix} z_0 & v_{z,0} & v_{h,0} & \theta_0 & m & \delta_{abk} \end{bmatrix}^\top, \tag{1.11}$$

where:

- z_0 is the initial altitude,
- $v_{z,0}$ is the initial vertical velocity,
- $v_{h,0} = \sqrt{v_{x,0}^2 + v_{y,0}^2}$ is the magnitude of the horizontal velocity,

- θ_0 is the pitch angle,
- m is the rocket mass,
- δ_{abk} is the airbrake deflection.

Some of these features are nonlinear combinations of the original state components (e.g., $v_{h,0}$), chosen to provide a more compact and physically meaningful representation of the flight condition while keeping the input dimension low.

1.2.5 Rationale for Feature Selection

The reduction from the full state $\mathbf{s}_0$ to the active feature vector $\mathbf{x}$ is driven by two main criteria:

1. **Impact on apogee**: variables with negligible influence on apogee within the considered flight envelope can be omitted without degrading prediction accuracy;
2. **Onboard measurability**: only quantities that can be measured or estimated reliably in flight are retained as inputs to the surrogate.

In particular:

- **Angular rates** (p_0, q_0, r_0) and yaw/roll angles (ψ_0, ϕ_0) were excluded on physical grounds — during a near-vertical coast their impact on apogee is expected to be second-order compared to altitude and vertical velocity — and because real-time estimation is affected by sensor noise and alignment uncertainties; their aggregate contribution is bounded, together with wind and aerodynamic scatter, by the validation error of Chapter 3;
- **Wind components** are not included explicitly in $\mathbf{x}$, since they cannot be measured with sufficient reliability onboard. Their effect is instead captured in the stochastic validation dataset by varying wind and aerodynamic coefficients while keeping $\mathbf{x}$ fixed;
- **Other geometric and atmospheric parameters** are assumed fixed in the nominal training set and are later perturbed only in the validation phase.

A formal upfront screening (e.g., via variance-based sensitivity analysis) was effectively replaced by an *a posteriori* analysis on a dedicated validation dataset. A more detailed discussion on sampling strategy, stochastic perturbations, and the construction of the validation dataset is provided in Section 2.4.

Chapter 2

Kriging-Based Surrogate Modeling Architecture

2.1 Overview of Kriging

Kriging, also known as Gaussian Process (GP) regression [3], is an interpolation technique that models the unknown function $\mathcal{F}(\mathbf{x})$ as a realization of a stochastic process. Originally developed for ore-grade estimation in geostatistics [4, 5], it has since become a standard tool for the design and analysis of computer experiments [6]. Unlike deterministic regression methods (e.g., polynomial least-squares or standard radial basis functions), Kriging provides not only a prediction but also a measure of uncertainty at each query point.

2.1.1 Gaussian Process Formulation

Following the formulation in Forrester et al. [7], the response $y(\mathbf{x})$ at any input $\mathbf{x} \in \mathbb{R}^d$ is modeled as the sum of a deterministic trend and a stochastic fluctuation:

$$y(\mathbf{x}) = \mu(\mathbf{x}) + Z(\mathbf{x}), \qquad \mu(\mathbf{x}) = \mathbf{f}(\mathbf{x})^T \boldsymbol{\beta}, \tag{2.1}$$

where:

- $\mu(\mathbf{x})$ is a low-order polynomial trend representing the global behavior of the function: $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^{n_\beta}$ collects the n_β polynomial basis functions and $\boldsymbol{\beta} \in \mathbb{R}^{n_\beta}$ the unknown regression coefficients;
- $Z(\mathbf{x})$ is a zero-mean Gaussian process that captures the local departures from the trend.

This regression-plus-correlation decomposition is known as *universal Kriging* and is the same structure adopted by DACE-style implementations [8]; the constant basis $\mathbf{f}(\mathbf{x}) = 1$ recovers ordinary Kriging with an unknown constant mean. Following the sensitivity study of Section 3.1.4, the nominal model adopts the full quadratic basis in the $d = 6$ normalized inputs,

$$\mathbf{f}(\mathbf{x}) = \left[\, 1,\ \ x_1, \ldots, x_d,\ \ x_1^2, x_1 x_2, \ldots, x_d^2 \,\right]^T, \tag{2.2}$$

i.e. the constant, the d linear monomials and the $d(d+1)/2$ quadratic monomials including cross-terms, for a total of $n_\beta = 28$ regressors.

The process $Z(\mathbf{x})$ is completely characterized by its covariance function (or kernel):

$$\mathrm{Cov}\big[Z(\mathbf{x}), Z(\mathbf{x}')\big] = \sigma^2 \, R(\mathbf{x}, \mathbf{x}'), \tag{2.3}$$

where σ^2 is the process variance and $R(\mathbf{x}, \mathbf{x}')$ is a correlation function satisfying $R(\mathbf{x}, \mathbf{x}) = 1$.

A standard choice is the *generalized exponential* (or power-exponential) correlation:

$$R(\mathbf{x}, \mathbf{x}') = \exp\left(- \sum_{j=1}^{d} \theta_j \, |x_j - x_j'|^{p_j} \right), \tag{2.4}$$

where $\theta_j > 0$ are the correlation parameters (one per input dimension) and $p_j \in [1, 2]$ control the smoothness of the process along each axis. Setting $p_j = 2$ for all j yields the squared-exponential (Gaussian) kernel, which assumes infinitely differentiable sample paths—a reasonable assumption for smooth physical responses such as apogee altitude.

2.1.2 Kriging Predictor and Variance

Given n training observations $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, define the response vector and the trend design matrix

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} \in \mathbb{R}^n, \qquad \mathbf{F} = \begin{bmatrix} \mathbf{f}(\mathbf{x}^{(1)})^\top \\ \vdots \\ \mathbf{f}(\mathbf{x}^{(n)})^\top \end{bmatrix} \in \mathbb{R}^{n \times n_\beta}, \tag{2.5}$$

and the $n \times n$ correlation matrix $\mathbf{R}$ with entries $R_{ij} = R(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$.

To regularize the (often ill-conditioned) correlation matrix, a small *nugget* λ is added to its diagonal, and every Kriging expression that follows uses the regularized matrix

$$\mathbf{R}_\lambda = \mathbf{R} + \lambda \mathbf{I} \tag{2.6}$$

in place of $\mathbf{R}$. The nugget improves numerical conditioning and slightly relaxes strict interpolation; its value is estimated jointly with the correlation parameters (Section 2.2).

At a new point $\mathbf{x}_*$, the Kriging predictor is the Best Linear Unbiased Predictor (BLUP):

$$\hat{y}(\mathbf{x}_*) = \mathbf{f}(\mathbf{x}_*)^\top \hat{\boldsymbol{\beta}} + \mathbf{r}(\mathbf{x}_*)^\top \mathbf{R}_\lambda^{-1} \left(\mathbf{y} - \mathbf{F}\hat{\boldsymbol{\beta}} \right), \tag{2.7}$$

where $\mathbf{r}(\mathbf{x}_*) \in \mathbb{R}^n$ is the correlation vector between $\mathbf{x}_*$ and all training points:

$$\mathbf{r}(\mathbf{x}_*) = \begin{bmatrix} R(\mathbf{x}_*, \mathbf{x}^{(1)}) \\ \vdots \\ R(\mathbf{x}_*, \mathbf{x}^{(n)}) \end{bmatrix}, \tag{2.8}$$

and $\hat{\boldsymbol{\beta}}$ is the generalized least-squares estimate of the trend coefficients:

$$\hat{\boldsymbol{\beta}} = \left(\mathbf{F}^\top \mathbf{R}_\lambda^{-1} \mathbf{F} \right)^{-1} \mathbf{F}^\top \mathbf{R}_\lambda^{-1} \mathbf{y}. \tag{2.9}$$

The associated prediction variance (mean squared error of the predictor) is

$$\hat{\sigma}^2(\mathbf{x}_*) = \hat{\sigma}^2 \left[1 - \mathbf{r}(\mathbf{x}_*)^\top \mathbf{R}_\lambda^{-1} \mathbf{r}(\mathbf{x}_*) + \mathbf{u}(\mathbf{x}_*)^\top \left(\mathbf{F}^\top \mathbf{R}_\lambda^{-1} \mathbf{F} \right)^{-1} \mathbf{u}(\mathbf{x}_*) \right], \tag{2.10}$$

with

$$\mathbf{u}(\mathbf{x}_*) = \mathbf{f}(\mathbf{x}_*) - \mathbf{F}^\top \mathbf{R}_\lambda^{-1} \mathbf{r}(\mathbf{x}_*), \tag{2.11}$$

where $\hat{\sigma}^2$ is the maximum-likelihood estimate of the process variance (derived below). The second term in Equation (2.10) is the reduction in uncertainty due to correlation with the observed data; the third accounts for the additional uncertainty introduced by estimating the trend coefficients from the same data, and collapses to the familiar ordinary-Kriging expression when the basis is constant. The variance vanishes at the training points up to the nugget (interpolation) and increases as $\mathbf{x}_*$ moves away from the data, providing a natural measure of model confidence.

2.2 Hyperparameter Estimation

The correlation parameters $\theta = [\theta_1, \ldots, \theta_d]^\top$ (and, in general, the smoothness exponents p_j) are unknown and must be estimated from the training data. Following Forrester et al. [7], we adopt a maximum likelihood approach.

2.2.1 Concentrated Log-Likelihood

Under the Gaussian process assumption, the joint distribution of the observations is

$$\mathbf{y} \sim \mathcal{N}\big(\mathbf{F}\boldsymbol{\beta},\, \sigma^2 \mathbf{R}_\lambda\big). \tag{2.12}$$

The log-likelihood function is

$$\ln L(\boldsymbol{\beta}, \sigma^2, \theta) = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln\Big(\sigma^2\Big) - \frac{1}{2}\ln|\mathbf{R}_\lambda| - \frac{1}{2\sigma^2}(\mathbf{y}-\mathbf{F}\boldsymbol{\beta})^\top \mathbf{R}_\lambda^{-1}(\mathbf{y}-\mathbf{F}\boldsymbol{\beta}). \tag{2.13}$$

For fixed θ, the optimal $\boldsymbol{\beta}$ and σ^2 can be found analytically by setting the partial derivatives to zero, yielding the generalized least-squares estimate of Equation (2.9) for $\hat{\boldsymbol{\beta}}$ and

$$\hat{\sigma}^2 = \frac{1}{n}(\mathbf{y}-\mathbf{F}\hat{\boldsymbol{\beta}})^\top \mathbf{R}_\lambda^{-1}(\mathbf{y}-\mathbf{F}\hat{\boldsymbol{\beta}}). \tag{2.14}$$

Substituting these back into Equation (2.13) and dropping constants, we obtain the *concentrated* (or profile) log-likelihood, which depends only on θ:

$$\ln L_c(\theta) = -\frac{n}{2}\ln\Big(\hat{\sigma}^2\Big) - \frac{1}{2}\ln|\mathbf{R}_\lambda|. \tag{2.15}$$

Maximizing $\ln L_c$ is equivalent to minimizing the loss function

$$\mathcal{L}(\theta) = n\ln\Big(\hat{\sigma}^2\Big) + \ln|\mathbf{R}_\lambda|, \tag{2.16}$$

which is the objective used in training. A standard alternative selects θ by minimizing the leave-one-out cross-validation (LOOCV) error, made practical by Rippa's identity [7, 9]; it is not adopted here.

2.2.2 Optimizer Selection

In the nominal model, the smoothness exponents are fixed to $p_j = 2$ for all dimensions, reflecting the assumption of a smooth (infinitely differentiable) apogee response with respect to the input features. The optimization therefore acts on the correlation length-scales $\theta \in \mathbb{R}^d_{>0}$ and on the nugget λ, with each θ_j parameterized through $\log_{10}\theta_j$ so that it can span several orders of magnitude.

The loss landscape $\mathcal{L}(\theta)$ (Equation (2.16)) is generally multimodal, yet it is smooth and can be evaluated cheaply through the Cholesky factor of $\mathbf{R}_\lambda$, which makes a gradient-based local solver an efficient choice. The concentrated log-likelihood is therefore minimized with **deterministic-start Sequential Quadratic Programming** (SQP) [10], using MATLAB's SQP solver with the exact analytical gradient of the loss supplied to it. A single deterministic midpoint start is adopted, which keeps training fully reproducible, while box constraints on $\log_{10}\theta_j$ and λ keep the solver away from ill-conditioned regions. The solver settings are summarized in Table 2.1.

The optimized nugget settles at the lower bound of its search interval rather than at an interior optimum. The resulting optimized length-scales and process variance are reported in Chapter 3 (Table 3.2).

Table 2.1: Deterministic-Start SQP Settings for Hyperparameter Optimization

Parameter	Value
Algorithm	SQP
First-order optimality tol.	10^{-6}
Step tolerance	10^{-6}
Max iterations per start	1000
Initialization strategy	Deterministic midpoint (single start)
$\log_{10}\theta_j$ bounds	$[-5, 3]$
Exponent p_j	fixed to 2 (nominal)
Nugget λ bounds	$[10^{-5}, 10^{-2}]$

Analytic gradient. The solver is supplied the exact gradient of the loss $\mathcal{L}$ (Equation (2.16)) rather than a finite-difference approximation. It is available in closed form and reuses the Cholesky factor of $\mathbf{R}_\lambda$ already formed to evaluate $\mathcal{L}$, so each gradient costs only back-substitutions. Writing $\boldsymbol{\alpha} = \mathbf{R}_\lambda^{-1}(\mathbf{y} - \mathbf{F}\hat{\boldsymbol{\beta}})$ for the residual weights and gathering the log-determinant and profiled-variance contributions into

$$\mathbf{W} = \mathbf{R}_\lambda^{-1} - \frac{\boldsymbol{\alpha}\,\boldsymbol{\alpha}^\mathsf{T}}{\hat{\sigma}^2}, \tag{2.17}$$

every hyperparameter derivative reduces to a single trace, $\partial\mathcal{L}/\partial\xi = \operatorname{tr}\!\big(\mathbf{W}\,\partial\mathbf{R}_\lambda/\partial\xi\big)$. With the pairwise gaps $\delta_{ij}^{(k)} = |x_i^{(k)} - x_j^{(k)}|$ and the power-exponential kernel of Equation (2.4), this gives

$$\frac{\partial\mathcal{L}}{\partial\theta_k} = -\sum_{i,j} W_{ij}\,R_{ij}\,\big(\delta_{ij}^{(k)}\big)^{p_k}, \tag{2.18}$$

$$\frac{\partial\mathcal{L}}{\partial p_k} = -\theta_k \sum_{i,j} W_{ij}\,R_{ij}\,\big(\delta_{ij}^{(k)}\big)^{p_k} \ln\delta_{ij}^{(k)}, \tag{2.19}$$

$$\frac{\partial\mathcal{L}}{\partial\lambda} = \operatorname{tr}(\mathbf{W}), \tag{2.20}$$

and the length-scale gradient is mapped to the optimized $\log_{10}\theta_k$ variable by $\partial\mathcal{L}/\partial(\log_{10}\theta_k) = \theta_k \ln(10)\,\partial\mathcal{L}/\partial\theta_k$. The exponent derivative in Equation (2.19) carries a $\ln\delta_{ij}^{(k)}$ factor that stays finite for every $p_k \in [1, 2]$ — coincident pairs, where $\delta_{ij}^{(k)} = 0$, contribute nothing — so the same gradient-based solver optimizes the smoothness exponents whenever they are freed rather than held at $p_k = 2$, which is exactly what the sensitivity study of Chapter 3 exploits.

2.3 Computational Considerations

2.3.1 Training Complexity

The dominant cost in Kriging training is the **Cholesky factorization** of the $n \times n$ correlation matrix $\mathbf{R}$, which requires $\mathcal{O}(n^3)$ floating-point operations. Forward/backward substitution to solve $\mathbf{L}\boldsymbol{\alpha} = \mathbf{y}$ and related systems costs $\mathcal{O}(n^2)$.

Each evaluation of $\mathcal{L}(\theta)$ requires one such factorization of $\mathbf{R}_\lambda$, so the total training cost scales as

$$T_{\text{train}} \sim N_{\text{eval}} \times \mathcal{O}(n^3), \tag{2.21}$$

where N_{eval} is the number of loss evaluations requested by the SQP solver across its iterations. Measured

training times for $n = 2000$ and the settings of Table 2.1 are reported in Section 3.4.

2.3.2 Prediction Complexity

After training, several quantities are precomputed and stored:

- the weight vector $\boldsymbol{\alpha} = \mathbf{R}_\lambda^{-1}(\mathbf{y} - \mathbf{F}\hat{\boldsymbol{\beta}})$;
- the trend coefficients $\hat{\boldsymbol{\beta}}$;
- the Cholesky factor $\mathbf{L}$ (for variance computation).

Prediction at a new point $\mathbf{x}_*$ then requires:

1. **Correlation vector**: compute $\mathbf{r}(\mathbf{x}_*)$ by evaluating n correlation functions: $\mathcal{O}(n \cdot d)$;
2. **Predictor mean**: $\hat{y}(\mathbf{x}_*) = \mathbf{f}(\mathbf{x}_*)^\top \hat{\boldsymbol{\beta}} + \mathbf{r}(\mathbf{x}_*)^\top \boldsymbol{\alpha}$: an $\mathcal{O}(n)$ dot product plus the evaluation of the n_β basis monomials;
3. **Predictor variance** (offline diagnostics only): Equation (2.10) requires the triangular solves $\mathbf{L}\mathbf{L}^\top \mathbf{v} = \mathbf{r}(\mathbf{x}_*)$, at $\mathcal{O}(n^2)$ per query.

The cost of a mean prediction is thus $\mathcal{O}(n \cdot d)$ per query, linear in the number of training points; the onboard control loop only evaluates the mean. For $n = 2000$ and $d = 6$, this translates to microsecond-scale inference (Section 3.4).

2.4 Sampling Strategy

The construction of the training dataset involves two main phases:

1. **Domain definition:** establishing the bounds of the input hypercube through stochastic flight simulations;
2. **Sample extraction:** selecting training points within the feasible region of the input space.

2.4.1 Input Domain Definition via Stochastic Simulations

The input domain for the surrogate model is an axis-aligned hypercube in $\mathbb{R}^d$ defined by lower and upper bounds on each feature:

$$\mathcal{X} = \prod_{j=1}^{d} [x_j^{\min}, x_j^{\max}]. \tag{2.22}$$

These bounds are determined by running a Monte Carlo campaign of full six-degree-of-freedom flight simulations with stochastic perturbations on:

- Thrust profile and total impulse;
- Vehicle inertias, mass and centre-of-mass offset;
- Launch rail angle;
- Aerodynamic coefficients (multiplicative uncertainties);
- Atmospheric wind (stochastic wind model).

Each simulation produces a complete trajectory from launch to apogee. By recording the reduced feature vector $\mathbf{x}$ (Equation (1.11)) at every time step after engine cutoff, we obtain an empirical envelope of physically realizable flight conditions. The bounds $x_j^{\min}$ and $x_j^{\max}$ are then set to the minimum and maximum values observed across all simulations.

This procedure ensures that the input domain covers the range of conditions the flight computer may encounter during actual flight, including off-nominal scenarios arising from manufacturing tolerances, atmospheric variability, and sensor uncertainties.

Validation dataset. The same stochastic simulations also serve as the *validation dataset*: because each includes parametric uncertainties absent from the reduced feature vector $\mathbf{x}$, the validation error measures robustness to unmodeled factors (Section 3.3.1).

2.4.2 Training Sample Extraction

A natural approach to populating the hypercube $\mathcal{X}$ is **Latin Hypercube Sampling (LHS)**, which provides space-filling designs with good coverage properties. However, LHS assumes that any point in the hypercube is a valid input, which is not the case here: the reduced features (altitude, vertical velocity, horizontal velocity, pitch, mass, airbrake deflection) are strongly coupled through the flight dynamics, and many combinations correspond to physically unrealizable trajectories.

Evaluating the target ODE at such infeasible points can lead to:

- Numerical failures (e.g., negative altitudes, divergent integration);
- Unphysical apogee values that corrupt the surrogate training;
- Wasted computational budget on meaningless samples.

Defining explicit constraints to carve out the feasible region is impractical due to the highly nonlinear and interlaced dependencies among the features.

Adopted approach: trajectory-based sampling. Instead of sampling the hypercube directly, training points are extracted from the stochastic simulation trajectories themselves:

1. Run N_{sim} stochastic simulations (here $N_{sim} = 1400$), varying the thrust profile and total impulse together with the other perturbations of Section 2.4.1, so that the post-cutoff states span the physically realizable flight envelope;
2. From each trajectory, randomly select N_{pts} time instants after engine cutoff (here $N_{pts} = 2$);
3. At each selected instant, record the reduced feature vector $\mathbf{x}$ and compute the corresponding apogee h_f by ballistic propagation of the state to apogee.

The apogee h_f in step 3 is obtained by integrating the coasting equations of motion of Section 1.2.1 (thrust-free, parachute-free) to apogee, with the wind set to zero. The initial condition is reconstructed from the reduced features, holding the yaw and roll angles fixed (Section 1.2.5) and leaving no body-frame lateral velocity, so the coast stays in a vertical plane. The propagation is therefore effectively three-degree-of-freedom, even though it integrates the six-degree-of-freedom equations.

The post-cutoff states form the sample pool, which is partitioned into 2000 training and 800 test points. The partition is performed at the trajectory level: samples drawn from the same simulation always fall on the same side of the split, so the test score is not inflated by correlated neighbours of states already seen in training. All samples are guaranteed to lie in physically plausible regions of the input space. Although they do not fill the entire hypercube uniformly, they concentrate in the regions that matter for flight control—namely, the manifold of realizable trajectories.

Normalization. Before training, each input feature is scaled to the unit interval $[0, 1]$ using the bounds from Section 2.4.1:

$$\tilde{x}_j = \frac{x_j - x_j^{min}}{x_j^{max} - x_j^{min}}. \tag{2.23}$$

The target apogee altitudes are instead standardized internally to zero mean and unit variance (z-score normalization) rather than rescaled to $[0, 1]$. All Kriging operations (correlation computation, hyperparameter optimization, prediction) are carried out in these normalized coordinates to improve numerical conditioning, and predictions are mapped back to physical units afterwards.

Chapter 3

Results and Performance Analysis

3.1 Nominal Kriging Model: Training and Hyperparameter Selection

3.1.1 Training Configuration

The nominal surrogate model is trained using the configuration summarized in Table 3.1.

The stochastic simulations perturb the parameters listed in Section 2.4.1: thrust and total impulse, vehicle inertias, mass and centre-of-mass offset, launch rail angle, aerodynamic coefficients and atmospheric wind.

The rocket mass is drawn uniformly over the admissible range of the vehicle and the airbrake deflection from $\delta_{abk} \in [0, 1]$. Across the retained trajectories, the observed apogee altitudes span $[359, 4670]$ m around a nominal target of roughly 3000 m.

3.1.2 Optimized Hyperparameters

The deterministic-start SQP minimization [10] of the concentrated log-likelihood (Equation (2.16)) yielded the correlation parameters and process variance reported in Table 3.2. The optimization converges monotonically to the reported minimum in about 25 iterations and, because the start point and the bounds are fixed, recovers the same optimum on every run.

Physical interpretation. In the parameterization of Equation (2.4), a larger θ_j makes the correlation decay faster along feature j. Since the quadratic trend absorbs the dominant smooth structure of the response, the length-scales describe the residual process rather than the full apogee map, and are best read together with the feature-importance analysis of Section 3.5.1. The residuals decorrelate fastest along δ_{abk} and $v_{z,0}$ (largest θ_j), while the very small θ_j of z_0 and m indicate that, once the trend is removed, the leftover variation along altitude and mass is smooth and long-range.

3.1.3 Dataset Convergence: Sample Size Selection

To verify that the chosen sample size n provides sufficient coverage without overfitting, independent Kriging models were trained on nested subsets of the stochastic-trajectory pool of increasing size n and each scored on the same held-out test set and the independent validation set. The resulting accuracy against n is listed in Table 3.3 and plotted in Figure 3.3(a).

At $n = 100$ the quadratic trend (28 coefficients) is still data-starved and the model is not yet usable; from $n = 200$ onwards the test error is already at the few-metre level and keeps improving with size. The validation error decreases in parallel and shows no degradation at higher n, so the model generalizes to the out-of-distribution scenarios (unmodeled wind, aerodynamic uncertainties). The reference $n = 2000$ uses the full trajectory pool and attains the best value of both curves, at a training cost that remains acceptable despite the $\mathcal{O}(n^3)$ scaling.

Table 3.1: Experimental Protocol and Nominal Model Configuration

Parameter	**Value**
Stochastic simulations	$N_{sim} = 1400$
Sample extraction strategy	Trajectory-based random sampling
Points per valid trajectory	$N_{pts} = 2$
Train/test split (by source trajectory)	$N_{traj,train} = 1000$, $N_{traj,test} = 400$ (of $N_{sim} = 1400$ total)
Training set size	$n = 2000$
Test set size	$n_{test} = 800$
Validation set	$n_{val} = 400$ true 6-DOF apogees, one per held-out *test* trajectory
Regression trend	Quadratic polynomial
Correlation function	Power-exponential ($p_j = 2$)
Hyperparameter optimization	Deterministic-start SQP (single midpoint start)
Nugget λ	$1.0e-05$

Table 3.2: Kriging hyperparameters (nominal model): fitted correlation parameters θ_j — larger values mean faster correlation decay along that feature — and estimated process variance. The pitch-angle feature is itself denoted θ_0 in the text.

Feature	θ_j	**Unit**	**Interpretation**
z_0	0.03401	—	Correlation decay rate along altitude
v_{z0}	5.947	—	Correlation decay rate along vertical velocity
v_{h0}	0.6805	—	Correlation decay rate along horizontal velocity
θ_0	1.363	—	Correlation decay rate along pitch
m	0.08931	—	Correlation decay rate along mass
δ_{abk}	14.56	—	Correlation decay rate along airbrake
$\hat{\sigma}^2$	12714	m^2	Estimated process variance

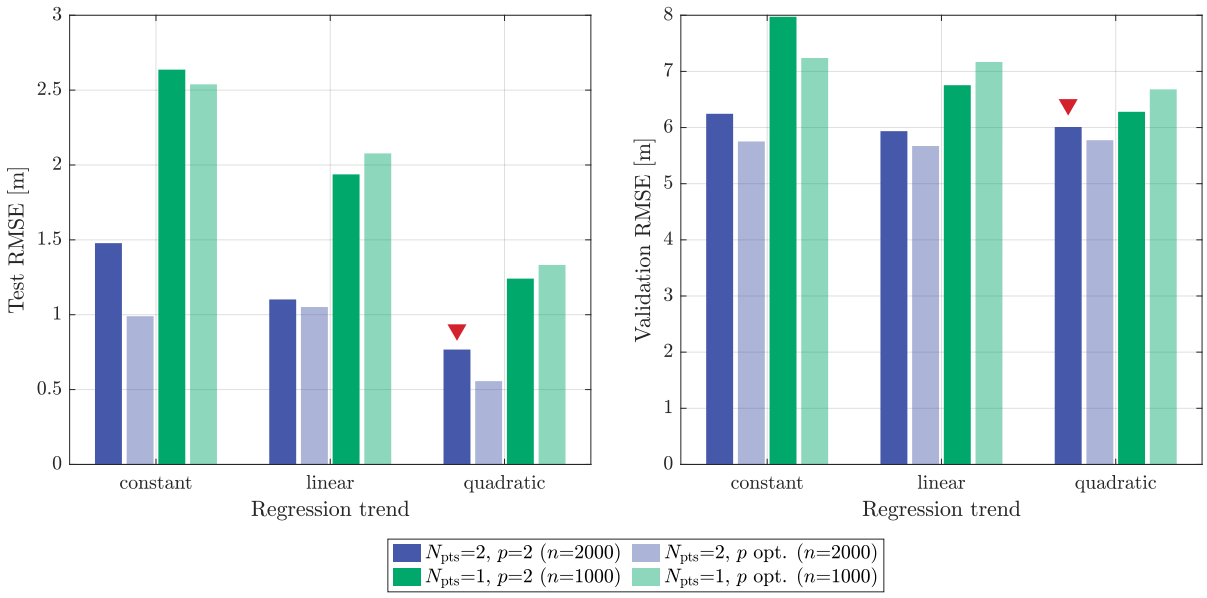


Figure 3.1: Sensitivity of test (left) and validation (right) RMSE to the regression trend, kernel exponents and samples-per-trajectory choice. Solid bars fix the Gaussian kernel ($p_j = 2$); light bars let the exponents be optimized in [1.5, 2]; the marker flags the nominal configuration.

3.1.4 Model Sensitivity Study

To justify the nominal configuration, twelve models were trained on the same campaign varying the regression trend degree, the correlation exponents (fixed Gaussian $p_j = 2$ versus p_j optimized in [1.5, 2]) and the number of samples extracted per trajectory ($N_{pts} \in \{1, 2\}$, at equal simulation cost). Every run shares the deterministic training procedure and the $N_{traj,train} = 1000$, $N_{traj,test} = 400$ trajectory partition of Table 3.1, but not the resulting sample counts: at $N_{pts} = 1$ training uses $n = 1000$ points and scores on 400 test points, while $N_{pts} = 2$ doubles both to $n = 2000$ and $n_{test} = 800$. The validation set is the same $n_{val} = 400$ held-out-trajectory points in every run. The outcome is summarized in Figure 3.1.

Three observations emerge:

- **Two samples per trajectory earn their cost:** at equal simulation budget, every $N_{pts} = 2$ configuration beats its $N_{pts} = 1$ counterpart on validation RMSE — the only metric compared at a matched sample count here, since the two arms' test sets hold 400 and 800 points respectively;
- **The Gaussian kernel is justified:** whenever the exponents are freed in [1.5, 2], the optimizer drives them back to $p_j \approx 2$; the residual gain over the fixed kernel (0.2–0.3 m of validation RMSE) lies within the statistical uncertainty of the 400-point validation set and costs a 5–10$\times$ longer training;
- **The trend choice matters mainly on the test metric:** at fixed kernel the quadratic trend roughly halves the test error with respect to the constant mean, but on validation the effect largely washes out — every $N_{pts} = 2$ configuration lands in the common 5.7–6.3 m band regardless of trend, dominated by the ballistic-propagation gap rather than by the trend choice.

The reference model of this chapter combines $N_{pts} = 2$, the quadratic trend and the fixed Gaussian kernel — the configuration with the best test error. On validation accuracy, however, its margin over the simpler trends is small, and a lower-order trend remains a legitimate choice where implementation simplicity matters.

A configuration could match the nominal on average yet degrade in the early coast, where the shutdown logic operates. Figure 3.2 therefore resolves the validation error of every configuration along the normalized coast phase t^*, separately for the two samples-per-trajectory datasets. All curves start widest just after cutoff, where the remaining flight is longest, and this early-coast region — where every configuration performs alike — dominates the aggregate validation figures. Through the mid and late coast the trend-bearing

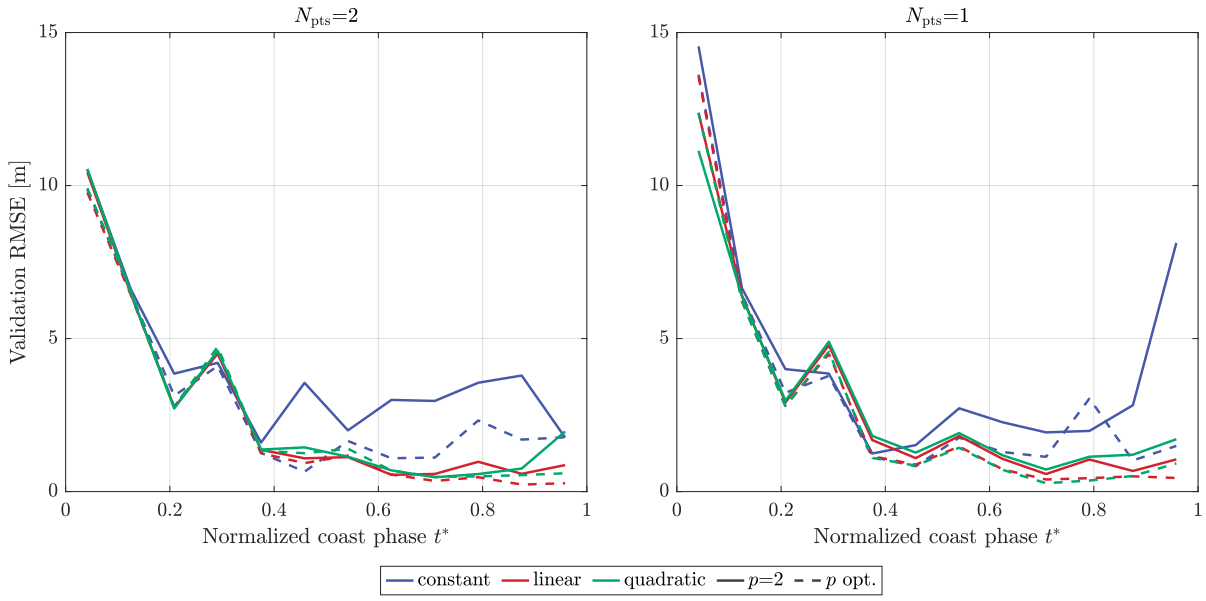


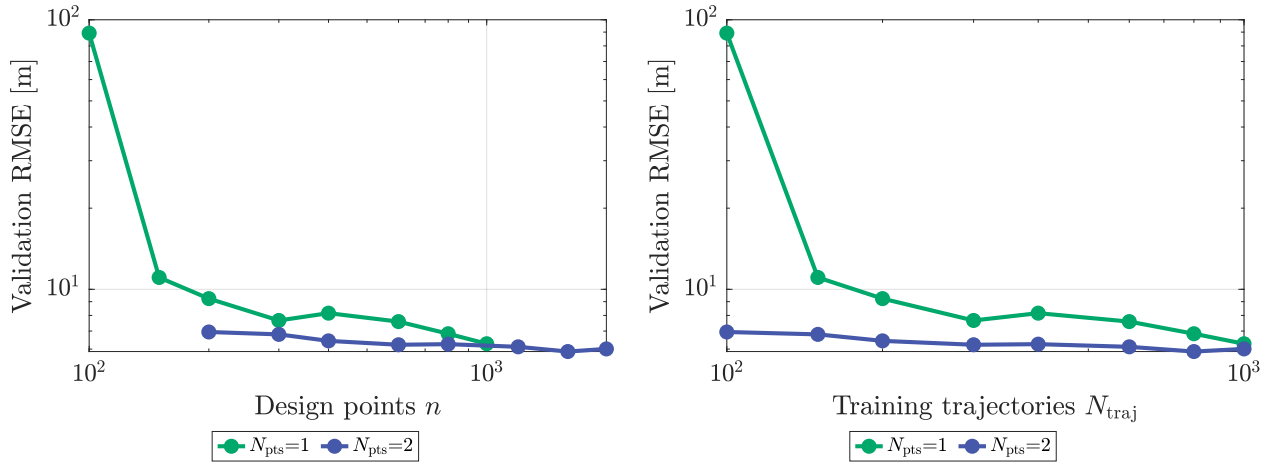
Figure 3.2: Binned validation RMSE against the normalized coast phase t^* for every sweep configuration, one panel per samples-per-trajectory dataset (12 equal-width bins, roughly 33 points each; bins with fewer than three points are dropped).

configurations cluster within about a metre of each other, while the fixed-kernel constant-mean model runs 1–3 m above them, with a larger excursion in the thinly populated final bin of the $N_{\text{pts}} = 1$ panel; the gap is visible but small against the early-coast error level, which is why it barely surfaces in the aggregate band of Figure 3.1.

3.1.5 Isolating the Two-Sample Effect from Training-Set Size

The sweep above changes N_{pts} by swapping dataset files, which moves n along with it and leaves the two effects entangled: the $N_{\text{pts}} = 2$ configurations are not merely trained with a second sample per trajectory — they are trained on twice as many points. A dedicated grid separates the two effects by crossing $N_{\text{pts}} \in \{1, 2\}$ with the number of training trajectories $N_{\text{traj,train}}$ explicitly, at the fixed Gaussian kernel and quadratic trend of the nominal configuration — the nominal-form slice of the design space — and scores every cell on the same $n_{\text{val}} = 400$ validation set. The test column likewise reuses the fixed held-out test split of each arm's dataset: across cells only the training set changes, shrunk by whole leading trajectories so that both arms draw from the same simulations at every $N_{\text{traj,train}}$. Table 3.3 lists every cell (latency is the warmed-up median per prediction, memory the mean-only footprint). Figure 3.3 plots validation RMSE against the two axes that matter. In panel (a) the abscissa is the training size n: points at the same abscissa were trained on the same amount of data, regardless of how it was collected. In panel (b) it is the number of simulated trajectories $N_{\text{traj,train}}$: points at the same abscissa cost the same simulation budget to generate.

At matched n (panel a) the gap between the two curves shrinks relative to panel (b) but does not close: $N_{\text{pts}} = 2$ stays ahead by 0.6–2.3 m at every shared size, so a second sample from an already-simulated trajectory is at least as informative here as a sample from a fresh one. At matched simulation budget (panel b) the gap widens, because $N_{\text{pts}} = 2$ also doubles n for free. The $N_{\text{pts}} = 2$ curve reaches its accuracy floor within a few hundred design points and stays there, while the $N_{\text{pts}} = 1$ curve is still descending, and non-monotone, out to the largest size tested. Two samples per trajectory therefore converge faster for a given amount of simulation, reinforcing the $N_{\text{pts}} = 2$ choice made above.



(a) Against training size n

(b) Against simulated trajectories $N_{traj,train}$

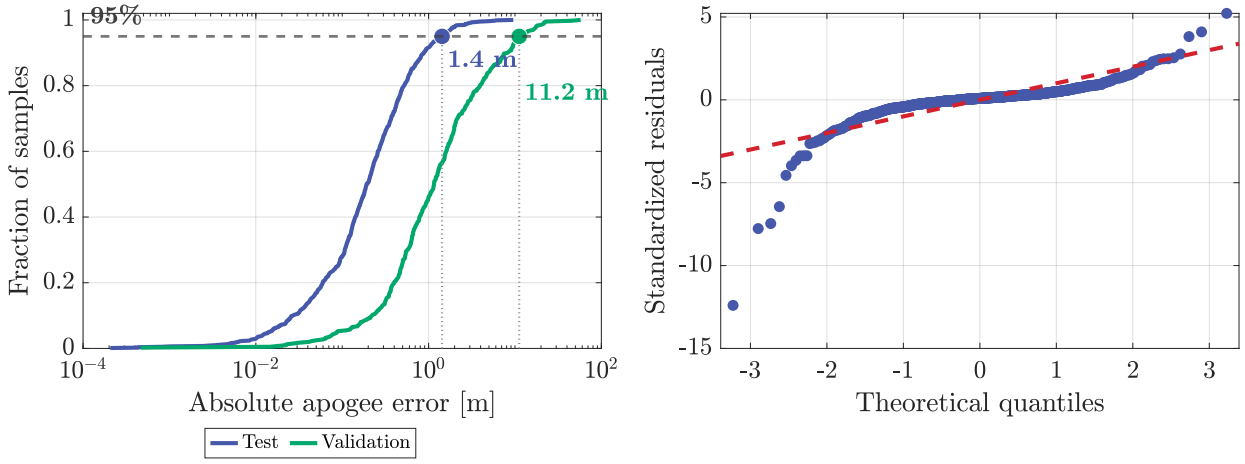
Figure 3.3: Validation RMSE of the nominal-form design-space slice (fixed Gaussian kernel, quadratic trend), one curve per samples-per-trajectory choice, against (a) training-set size and (b) the number of simulated trajectories.

Table 3.3: Design-Space Grid: Accuracy, Latency and Memory versus Training Trajectories and Samples per Trajectory (Fixed Gaussian Kernel, Quadratic Trend)

$N_{traj,train}$	N_{pts}	n	**$RMSE_{test}$ [m]**	**$RMSE_{val}$ [m]**	**Latency [µs]**	**Memory [KB]**
100	1	100	60.54	89.22	1.8	6
150	1	150	5.14	11.06	2.6	8
200	1	200	3.60	9.24	3.5	11
300	1	300	2.31	7.66	5.1	17
400	1	400	2.38	8.16	6.7	22
600	1	600	2.10	7.59	10.3	33
800	1	800	1.98	6.84	13.3	44
1000	1	1000	1.24	6.28	17.6	55
100	2	200	3.31	6.95	3.5	11
150	2	300	2.41	6.80	5.2	17
200	2	400	2.01	6.44	6.6	22
300	2	600	1.51	6.23	10.1	33
400	2	800	1.28	6.26	13.4	44
600	2	1200	1.24	6.12	19.9	66
800	2	1600	1.25	5.88	24.3	88
1000	2	2000	0.77	6.01	33.9	110

Table 3.4: Model Performance on Test Set (Nominal Model)

Metric	Value	Unit
RMSE	0.77	m
Mean Absolute Error (MAE)	0.39	m
R^2	1.0000	—



(a) Absolute-error CDF

(b) Q–Q plot

Figure 3.4: Empirical CDF of the absolute apogee error (test and validation sets, 95th percentiles flagged) and test-set residual Q–Q plot against a normal distribution (residuals standardized by the test-set sample standard deviation).

3.2 Accuracy Analysis

3.2.1 Test Set Performance Metrics

Table 3.4 reports the accuracy of the nominal model on the held-out test set, whose targets are generated by the same ballistic propagation used for training.

An RMSE of 0.77 m, over apogees spanning [359, 4670] m with a target near 3000 m, corresponds to a relative error well below 0.1%: on its own training-target distribution the surrogate reproduces the ballistic apogee almost exactly ($R^2 = 1.0000$).

3.2.2 Prediction Error Distribution

The percentiles read directly off the CDF: 95% of the test predictions fall within 1.43 m of the ballistic target, and 95% of the validation predictions within 11.19 m of the true six-DOF apogee. The residuals are tightly concentrated about zero and approximately Gaussian in the bulk, as the Q–Q plot shows, but the tails are heavier than Gaussian (Section 3.5.3).

3.2.3 Parity Plots

On the test set the predictions lie essentially on the parity line across the whole apogee range. The validation panel shows the same structure with a visibly wider scatter — the ballistic-versus-six-DOF gap discussed in Section 3.3.1 — yet no curvature or systematic offset, so the extra error behaves like noise around an unbiased predictor. How literally the $\pm 2\hat{\sigma}$ bands themselves can be trusted is quantified in Section 3.5.3.

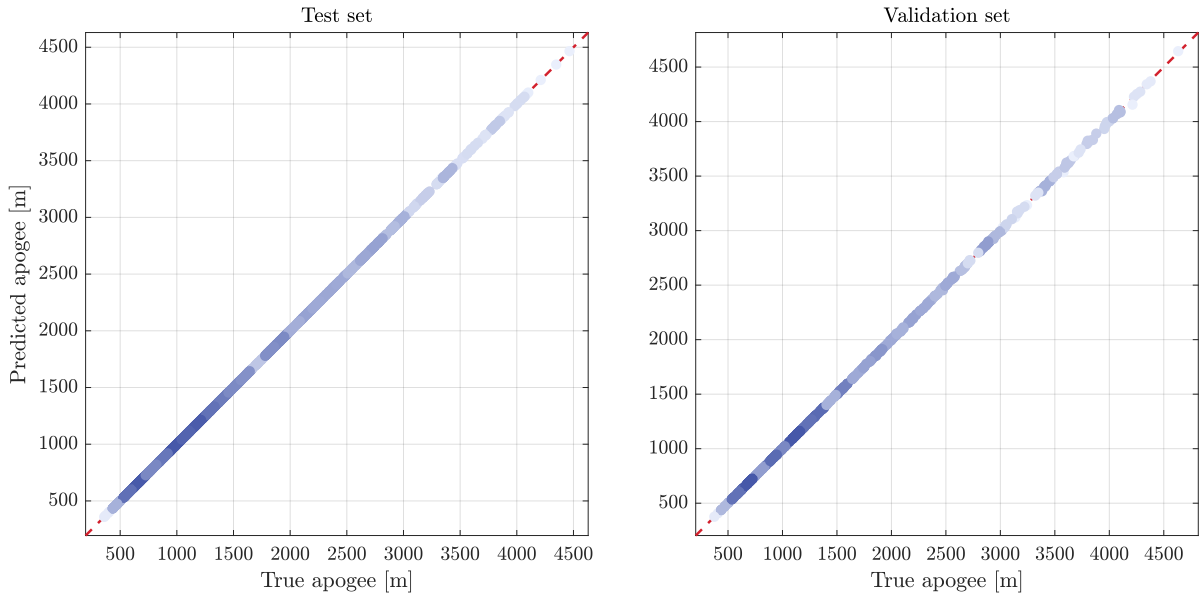


Figure 3.5: Predicted versus true apogee on the test set (left) and the held-out-trajectory validation set (right); grey bars show $\pm 2\hat{\sigma}$ on an evenly spaced subsample, darker points lie in denser regions.

3.2.4 Surrogate Response Maps

The maps in Figures 3.6 and 3.7 look directly at the learned response. The $(v_{z,0}, \delta_{abk})$ map is the view the control law cares about: at any coast state it quantifies how much apogee the airbrakes can still remove. The one-dimensional slices cut the response along each input, with all other features held at the training mean. Inputs are shown in the normalized coordinates of Equation (2.23), where 0 and 1 are the bounds of the sampled envelope. Altitude and vertical velocity follow the z-down NED convention of Section 1.2.1, so their lower bound is the highest altitude and the fastest ascent.

The response is smooth and monotone in the airbrake extension across the sampled envelope, with an authority of a few hundred metres at energetic states. At the scale of the response the $\pm 2\hat{\sigma}$ bands are barely visible, so rather than inflating them, each panel carries a narrow strip below it with the band half-width $2\hat{\sigma}(x)$ in metres. The predicted variance grows where the trajectory manifold leaves few samples, marking the region where the surrogate extrapolates.

3.3 Robustness Under Parametric Uncertainty

3.3.1 Validation Against Stochastic Simulations

A stricter assessment of robustness uses the independent *validation set* of 400 samples, drawn exclusively from held-out trajectories that contribute nothing to training, whose targets are the *true* apogees of the full ascent trajectories rather than the ballistic propagation used in training. This set therefore measures two error sources at once: the surrogate approximation itself, and the gap between the simplified ballistic target and the full six-DOF simulation (including wind and aerodynamic uncertainties not represented in the reduced feature vector).

For each validation trajectory:

1. Full six-DOF simulation with aerodynamic uncertainties and wind;
2. Extract the reduced feature vector $\mathbf{x}$ at the prediction time;
3. Predict apogee with the trained Kriging model, $\hat{h}_f = \text{Kriging}(\mathbf{x})$;

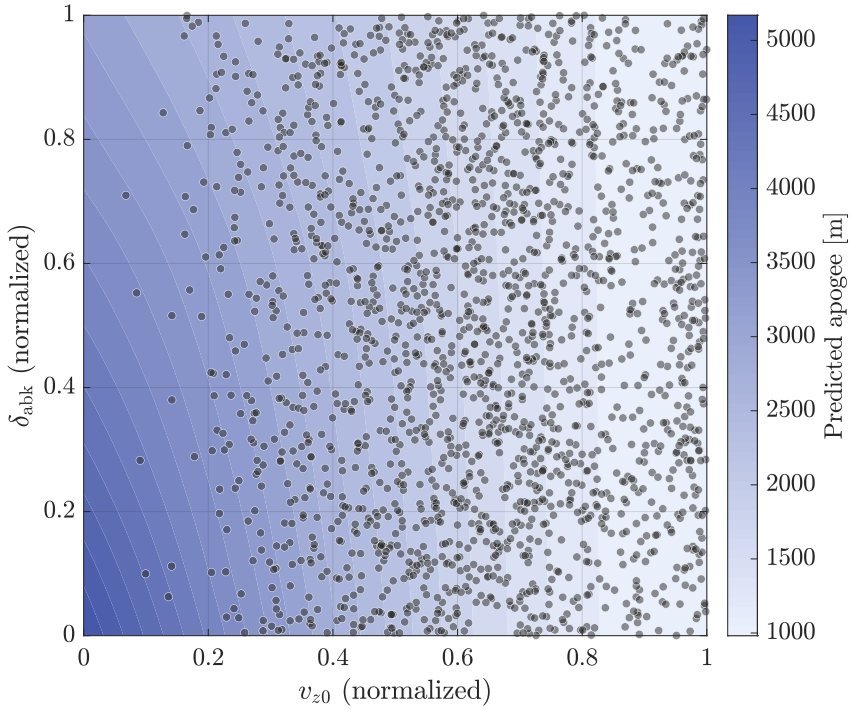


Figure 3.6: Predicted apogee over normalized vertical velocity and airbrake extension, other inputs at the training mean; grey dots project the training samples.

Table 3.5: Model Performance on Validation Set (Stochastic Scenarios)

Metric	**Value**	**Unit**
RMSE (validation)	6.01	m
MAE (validation)	2.93	m
R^2 (validation)	1.0000	—

4. Compare against the true apogee of the full simulation.

The validation RMSE (6.01 m) is larger than the test RMSE (0.77 m), as expected: it absorbs both the unmodeled wind and aerodynamic scatter and the ballistic-versus-full-simulation modelling gap. Even so, at 6.01 m against apogees of several kilometres the relative error remains a fraction of a percent ($R^2 = 1.0000$), which confirms that the surrogate stays accurate under realistic flight variability.

Figure 3.8 resolves this error along the coast: each validation state is extracted at a different normalized instant t^* between engine cutoff ($t^* = 0$) and apogee ($t^* = 1$).

The binned mean stays close to zero along the whole coast — the predictor is essentially unbiased at every phase — while the spread is widest for states taken just after cutoff, where the remaining flight is longest and the unmodeled disturbances have the most time to act, and shrinks as the state approaches apogee. For the onboard use case this is the favourable direction: predictions keep improving as the maneuver decision point nears.

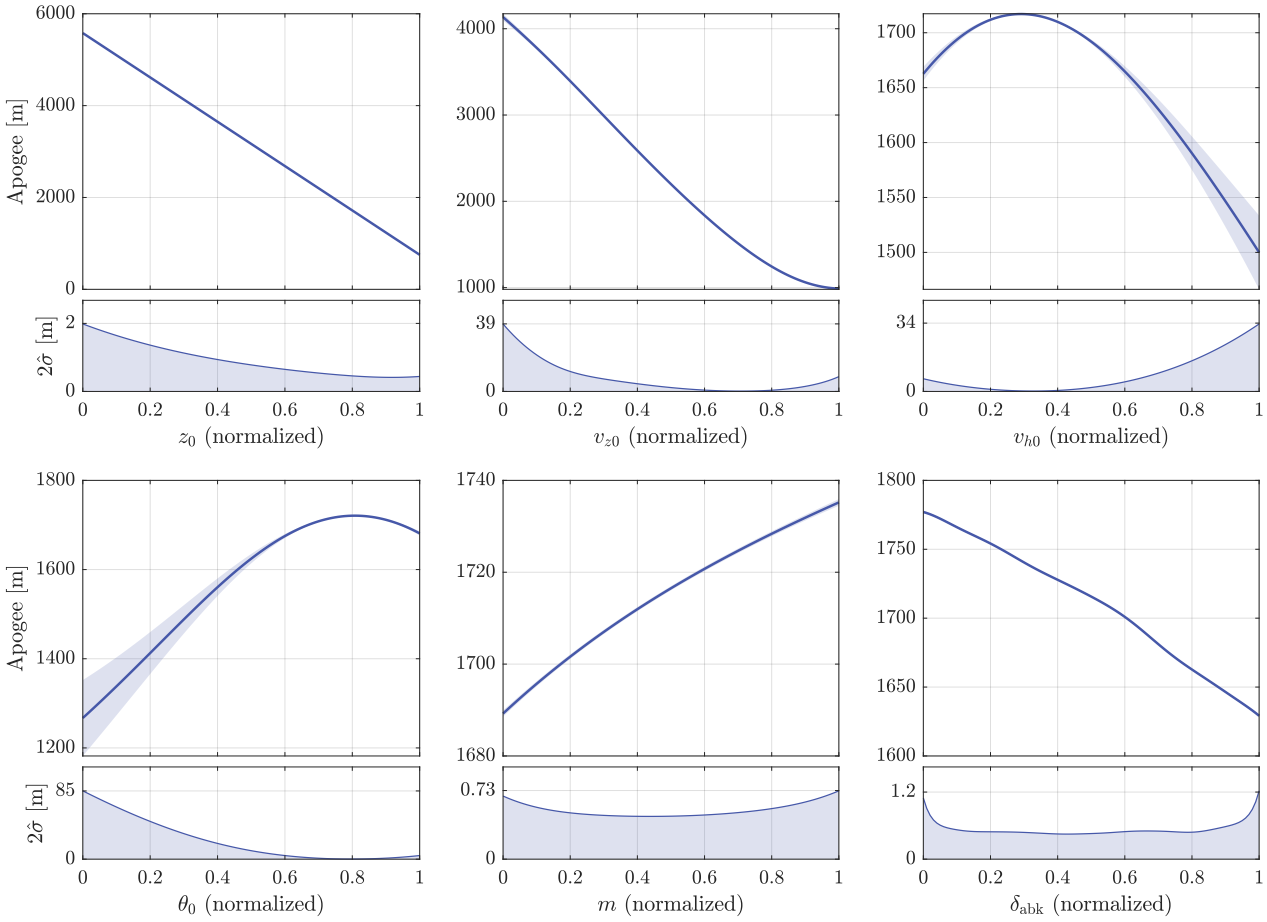


Figure 3.7: One-dimensional response slices along the normalized inputs ($\pm 2\hat{\sigma}$ bands), other inputs at the training mean; the strip below each panel plots the band half-width $2\hat{\sigma}(x)$ in metres.

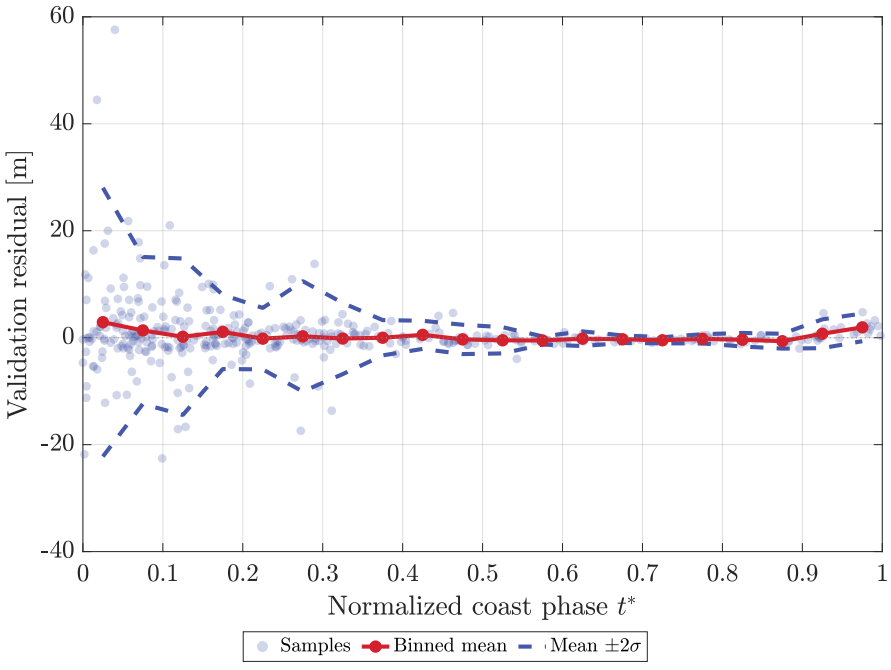


Figure 3.8: Validation residuals versus the normalized coast phase at which the state was extracted: signed binned mean with mean $\pm 2\sigma$ spread curves.

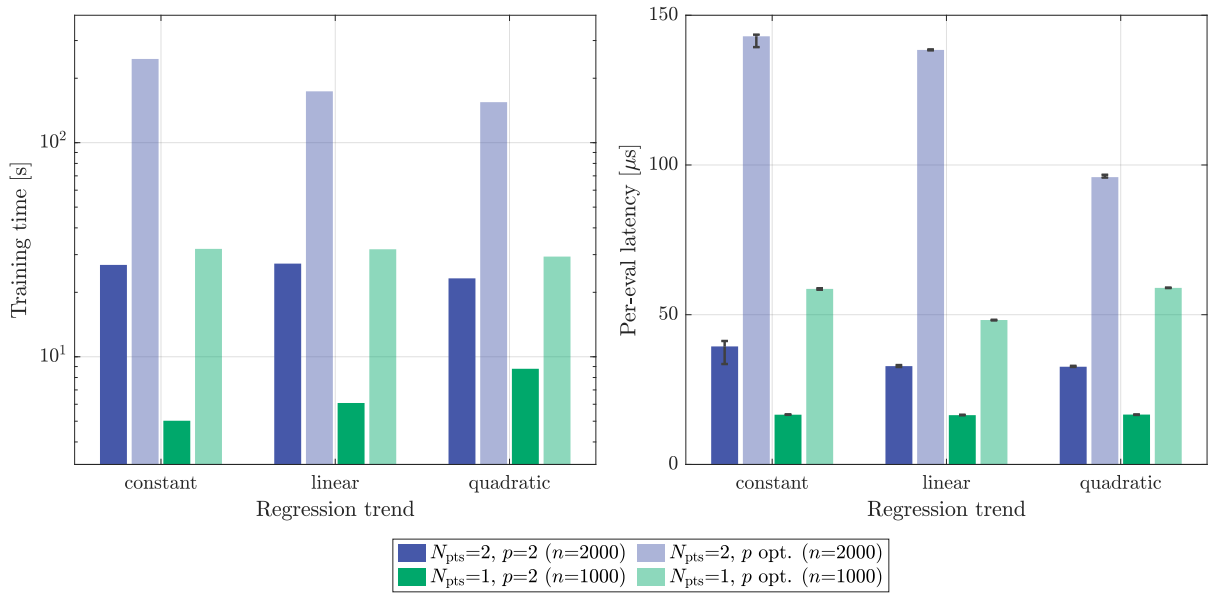


Figure 3.9: Computational cost of every sweep configuration: measured training time (log scale) and warmed-up prediction latency, as median with interquartile bars over repeated queries.

3.4 Computational Performance

3.4.1 Training Cost

Training is performed once, offline. It is dominated by the Cholesky factorizations of the $n \times n$ regularized correlation matrix $\mathbf{R}_\lambda$ required at each loss evaluation (Section 2.3.1). Rather than quoting a single runtime, Figure 3.9 reports the measured wall-clock training time of every configuration of the sensitivity study (Section 3.1.4): the fixed-kernel models, the nominal one included, sit at the cheap end of the range, while freeing the correlation exponents multiplies the cost by the 5–10$\times$ factor already quoted there. Part of that spread is not the kernel at all: the two colour groups in the figure also differ by a factor of two in n ($N_{\text{pts}} = 2$ trains on $n = 2000$, $N_{\text{pts}} = 1$ on $n = 1000$), and since fitting scales as $\mathcal{O}(n^3)$ and prediction as $\mathcal{O}(n)$ (Section 2.3.1), most of the gap between the two colours, not just between shades within one, is a training-set-size effect rather than a trend or kernel effect. Since training never runs on the flight computer, its absolute runtime is not a real-time concern.

3.4.2 Inference Cost

Throughout this report, prediction latency is the warmed-up median time per prediction: 10^4 queries are issued in a single vectorised call, the elapsed time is divided by the query count, and the median over seven repetitions is taken. Measuring over batched queries isolates the cost of the predictor's arithmetic from the interpreter's call overhead, an artefact of the MATLAB prototyping environment that a compiled on-board implementation would not pay. Under this metric the nominal surrogate takes 33.9 µs per prediction — the same measurement reported in the last row of Table 3.3.

The natural baselines are the physics-based predictors the surrogate replaces: the full six-DOF ascent simulation and the simplified ballistic propagation used to generate the training targets. Both were measured on the same machine, in strictly serial execution: propagating a single state ballistically to apogee takes 65.1 ms per evaluation (median of 50 calls), and a full six-DOF ascent simulation 3.53 s per run (mean of 3 runs, including the post-cutoff extraction pipeline, not just the ascent integration). For the baselines the batching above is immaterial: each evaluation is an independent ODE integration whose millisecond-to-second cost dwarfs any call overhead. The surrogate is about 1900$\times$ faster than the ballistic propagation and

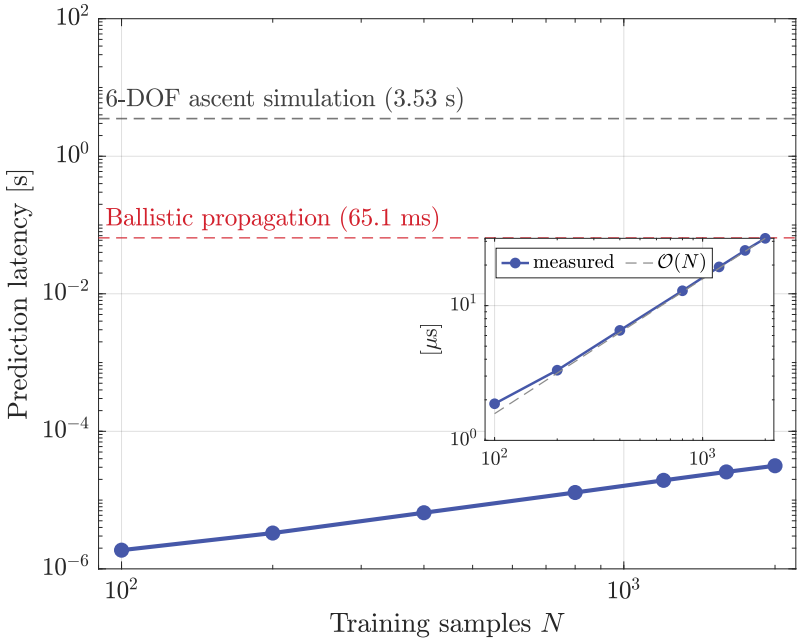


Figure 3.10: Warmed-up prediction latency versus training-set size, against the measured physics-based baselines; the inset repeats the surrogate curve against an $\mathcal{O}(N)$ reference.

about 100000× faster than the full simulation, well within the per-cycle budget of the onboard control loop; Figure 3.10 places both baselines against the surrogate latency across model sizes.

Latency grows roughly linearly with the training-set size, as the inset of Figure 3.10 shows against an $\mathcal{O}(N)$ reference: even the largest model remains far below the ballistic alternative, so accuracy can be bought with data without threatening the real-time budget. Figure 3.9 repeats the same warmed-up measurement for every configuration of the sensitivity study, as median with interquartile bars over repeated queries: the run-to-run spread is attached to each estimate, and no candidate configuration, nominal included, comes close to the physics-based baselines.

Memory-wise, the stored predictor holds the $n \times d$ training inputs, the dual coefficient vector and the quadratic-trend coefficients, i.e. $(2000 \times 6 + 2000 + 28)$ double-precision values, about 110 KB. That figure is the *mean-only* footprint: it excludes the $n \times n$ Cholesky factor $\mathbf{L}$ that Section 2.3.2 lists among the quantities stored for the variance computation, and which dominates the 15.8 MB stored model. A deployment that only needs the point prediction can ship at the smaller figure; keeping $\hat{\sigma}^2(\mathbf{x})$ available on-board costs the full Cholesky factor instead.

3.4.3 Design-Space Trade-off and Deployment Recommendation

Validation RMSE saturates after a few hundred design points: past that size the surrogate is no longer the limiting error source, and the residual is dominated by the ballistic-versus-six-DOF modelling gap of Section 3.3.1. Cost, by contrast, keeps growing with n, so a smaller model than the reference $n = 2000$ is a legitimate deployment choice; the configurations to choose from are already listed in Table 3.3, and which one to pick depends on whether loop time or memory footprint is the binding resource on the target hardware.

Two examples bracket the choice. Measured against the reference model's row (Table 3.3, last row), a fixed-Gaussian model at $n = 600$ (6.23 m validation RMSE, 10.1 µs, 33 KB) cuts both latency and footprint to about a third. A model at $n = 300$ with the correlation exponents optimized (6.18 m, 14.7 µs, 17 KB) trims memory to about a sixth but latency only to about two-fifths, because an optimized-exponent kernel costs more per training point to evaluate. Both sit within 0.25 m (about 4%) of the reference model's validation

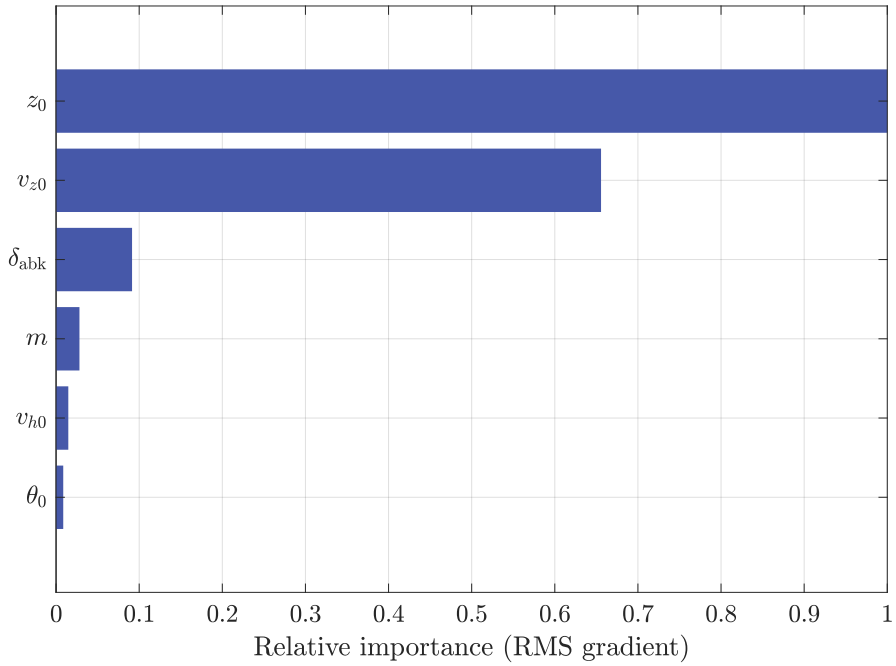


Figure 3.11: Feature-importance ranking of the nominal model: RMS Kriging gradient over the training set, computed in the normalized input space and scaled to a maximum of one.

RMSE, inside the 5.7–6.3 m plateau band, and both use $N_{pts} = 2$.

3.5 Sensitivity and Uncertainty Analysis

3.5.1 Feature Importance via Gradient-Based Analysis

To quantify the influence of each input feature on apogee, finite-difference approximations of the Kriging gradient are computed across the training domain:

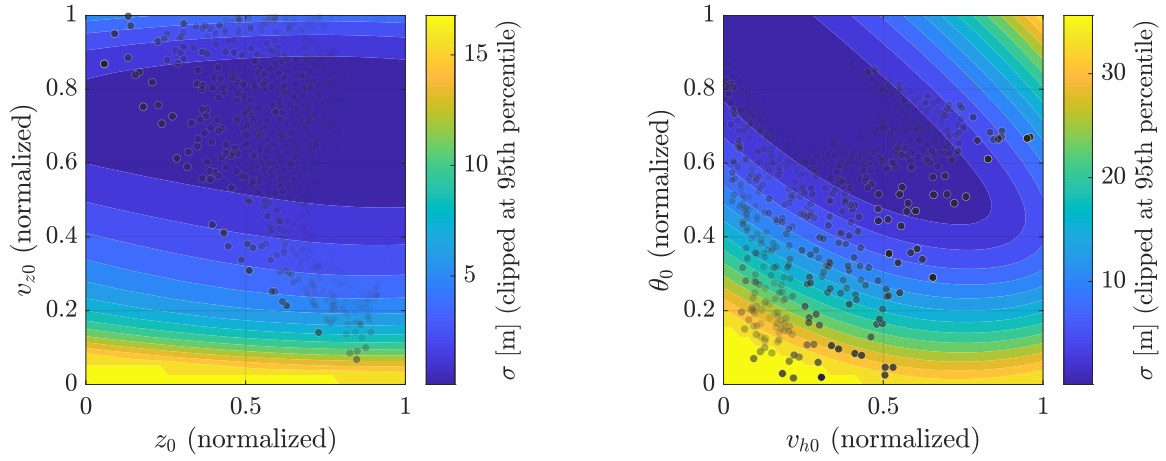
$$\left.\frac{\partial \hat{y}}{\partial x_j}\right|_{\mathbf{x}} \approx \frac{\hat{y}(\mathbf{x} + \epsilon \mathbf{e}_j) - \hat{y}(\mathbf{x} - \epsilon \mathbf{e}_j)}{2\epsilon} \tag{3.1}$$

A feature-importance score is then the RMS gradient over the training domain:

$$\text{Importance}_j = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \left(\left.\frac{\partial \hat{y}}{\partial x_j}\right|_{\mathbf{x}^{(i)}} \right)^2}. \tag{3.2}$$

Interpretation. The ranking places the altitude z_0 and the vertical velocity $v_{z,0}$ at the top, with the airbrake deflection δ_{abk} next, while the horizontal velocity $v_{h,0}$, the mass m and the pitch angle θ_0 contribute far less. This matches the physics of a near-vertical ballistic coast, where the residual potential and kinetic energy dominate the outcome, and complements the residual length-scales of Table 3.2 (Section 3.1.2). It also carries practical consequences:

- Sensor accuracy and calibration should be prioritized for the dominant states (vertical velocity and altitude);
- The least influential features are candidates for removal in future, leaner model variants.



(a) $\hat{\sigma}$ over z_0, $v_{z,0}$

(b) $\hat{\sigma}$ over $v_{h,0}$, θ_0

Figure 3.12: Prediction uncertainty $\hat{\sigma}$ across two 2D slices of the normalized input domain, other inputs at the training mean; grey dots are training samples projected onto the slice plane, and the colour scale is clipped at the 95th percentile of $\hat{\sigma}$ within each window.

3.5.2 Prediction Uncertainty Map

The Kriging model provides pointwise variance estimates $\hat{\sigma}^2(\mathbf{x})$ everywhere in the input domain. Figure 3.12 visualizes this uncertainty landscape over two representative 2D slices.

Uncertainty is lowest in the core training region, where samples are dense, and grows towards the boundary of the feasible region (extrapolation zone); without the percentile clipping of the colour scale, this boundary growth would flatten the interior structure. The asymmetric pattern — clearest in the $(v_{h,0}, \theta_0)$ slice — reflects the clustering of the data along realistic trajectories rather than a uniform fill of the hypercube. This landscape is also the quantity that sequential infill criteria build on [11]; here it is used diagnostically, to mark where the surrogate should not be trusted.

3.5.3 Variance Calibration

A variance map is only useful if its magnitude can be trusted. Figure 3.13 checks this on the test set from two angles: the standardized residuals $r/\hat{\sigma}$ against the standard normal density, and a reliability curve tracing the empirical central coverage against its nominal level over the whole range of nominal levels. The curve runs below the diagonal of perfect calibration over the whole range of nominal levels, and the shortfall is largest in the upper range that matters for interval design: at the flagged 95% level the $\pm 2\hat{\sigma}$ interval empirically covers 79.9% of the test targets. The predicted variance is optimistic, with residual tails clearly heavier than Gaussian. Restoring nominal coverage requires inflating $\hat{\sigma}$ by roughly $1.5\times$ — the dash-dotted reliability curve in Figure 3.13, which runs close to the diagonal — so a control law should treat the raw bands as sharp relative indicators of where the model is more or less sure, and apply that safety factor before using them as absolute 95% bounds.

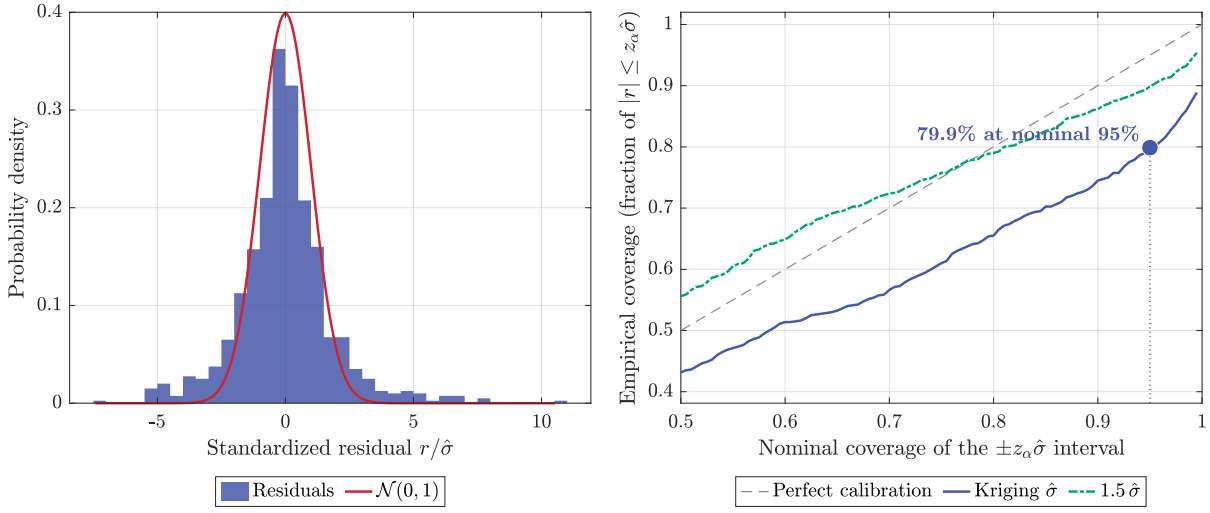


Figure 3.13: Kriging variance calibration on the test set: standardized residuals against $\mathcal{N}(0,1)$ (left) and reliability curve of empirical versus nominal central coverage, with the diagonal of perfect calibration, the 95% level flagged, and the same check with $\hat{\sigma}$ inflated by $1.5\times$ (right).

Chapter 4

Conclusions and Future Perspectives

4.1 Summary and Key Results

This work developed and validated a Kriging surrogate for real-time apogee prediction:

1. a predictor fast enough for on-board flight control, with a prediction latency of a few tens of microseconds (Table 4.1) and a test RMSE of 0.77 m;
2. pointwise variance estimates, empirically optimistic by roughly 1.5× (Section 3.5.3), usable by a control law as relative confidence indicators;
3. validation against independent full six-DOF stochastic simulations, which isolates the ballistic-to-six-DOF modelling gap from the surrogate error.

The headline performance indicators are collected in Table 4.1.

Table 4.1: Summary of Key Performance Indicators

Metric	**Value**
Prediction latency	33.9 µs
Test RMSE	0.77 m
Validation RMSE	6.01 m
Memory footprint (mean-only)	$\approx$ 110 KB

The deployed footprint is the mean-only figure of Section 3.4.2: about 110 KB, excluding the Cholesky factor needed only if the variance is computed on-board. A design-space study (Section 3.4.3) shows the reference model is not the only viable size: a fixed-Gaussian model at $n = 600$ (6.23 m, 10.1 µs, 33 KB) and an optimized-exponent model at $n = 300$ (6.18 m, 14.7 µs, 17 KB) match the reference validation accuracy (6.01 m) to within a few percent at a fraction of the cost, and on constrained hardware they can be the preferable deployment.

4.2 Implications for the Flight Software

The surrogate is intended to support the engine-shutdown (SDA) and air-brake (ABK) algorithms of Hydra. It is presented as a component integrated in the control stack, not as a flight-proven one. Concretely, the flight software gains:

- A real-time apogee estimate for engine-shutdown optimization;
- A common predictor that the air-brake control law can share, as an alternative to offline lookup tables, with a variance attached to each prediction;
- Reduced reliance on ground-precomputed trajectories;

- A validated lighter-weight fallback (Section 3.4.3) for hardware where loop time or memory is tightly constrained.

4.3 Limitations and Caveats

Several limitations should be kept in mind:

- **Validity region:** predictions are trustworthy only while the input features remain inside the sampled domain; uncertainty grows quickly in the extrapolation zone;
- **Unmodeled disturbances:** wind and aerodynamic scatter are not part of the reduced feature vector, so their effect appears as the larger validation error (6.01 m) rather than being predicted explicitly;
- **Training target:** the training labels come from a simplified ballistic propagation; the residual gap to the full six-DOF apogee is quantified by the validation set (Section 3.3.1) but not removed;
- **Variance calibration:** the predicted $\hat{\sigma}$ is optimistic (empirical coverage of about 79.9% at a nominal 95%); absolute confidence bounds require the inflation factor quantified in Section 3.5.3;
- **Configuration specificity:** the model is tied to the specific vehicle configuration and nominal atmosphere used to generate the dataset, and would need retraining for a different vehicle.

4.4 Future Work

Three directions follow directly from the findings above:

- *Multi-fidelity modeling*: a co-Kriging framework [2] could combine the cheap ballistic targets with a small number of full six-DOF simulations, directly narrowing the ballistic-to-full modelling gap that dominates the validation error;
- *Wind as a feature*: promoting wind magnitude and direction from unmodeled disturbances to explicit inputs would let the surrogate resolve part of the current validation error rather than absorbing it into the variance;
- *Closed-loop integration*: using the predictions and their variance in the actuator commands, with measurement feedback, and ultimately validating the stack on test flights.

Bibliography

[1] N. V. Queipo, R. T. Haftka, W. Shyy, T. Goel, R. Vaidyanathan, and P. K. Tucker, "Surrogate-based analysis and optimization," *Progress in Aerospace Sciences*, vol. 41, no. 1, pp. 1–28, 2005.

[2] A. I. J. Forrester and A. J. Keane, "Recent advances in surrogate-based optimization," *Progress in Aerospace Sciences*, vol. 45, no. 1–3, pp. 50–79, 2009.

[3] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA: MIT Press, 2006.

[4] D. G. Krige, "A statistical approach to some basic mine valuation problems on the Witwatersrand," *Journal of the Chemical, Metallurgical and Mining Society of South Africa*, vol. 52, no. 6, pp. 119–139, 1951.

[5] G. Matheron, "Principles of geostatistics," *Economic Geology*, vol. 58, no. 8, pp. 1246–1266, 1963.

[6] J. Sacks, W. J. Welch, T. J. Mitchell, and H. P. Wynn, "Design and analysis of computer experiments," *Statistical Science*, vol. 4, no. 4, pp. 409–423, 1989.

[7] A. I. J. Forrester, A. Sóbester, and A. J. Keane, *Engineering Design via Surrogate Modelling: A Practical Guide*. Chichester, UK: Wiley, 2008.

[8] S. N. Lophaven, H. B. Nielsen, and J. Søndergaard, "DACE – a MATLAB Kriging toolbox, version 2.0," Informatics and Mathematical Modelling, Technical University of Denmark, Kgs. Lyngby, Denmark, Tech. Rep. IMM-TR-2002-12, 2002.

[9] S. Rippa, "An algorithm for selecting a good value for the parameter c in radial basis function interpolation," *Advances in Computational Mathematics*, vol. 11, pp. 193–210, 1999.

[10] J. Nocedal and S. J. Wright, *Numerical Optimization* (Springer Series in Operations Research and Financial Engineering), 2nd ed. New York: Springer, 2006.

[11] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient global optimization of expensive black-box functions," *Journal of Global Optimization*, vol. 13, no. 4, pp. 455–492, 1998.